

STRATEGY · ARCHITECTURE · HARDWARE REPORT

Agentic Orchestration Platform

A Multi-Agent Code Generation System:
Full Build Plan, Costs, and the RTX 1650 4GB Bottleneck

Prepared for

Local development environment — ASUS TUF F15
GPU: NVIDIA RTX 1650, 4GB VRAM

Status

Ready for implementation
Recommended path: Cloud-hybrid (Local + Free GPU)

Table of Contents

1. Executive Summary
2. The Hardware Reality Check — Why the RTX 1650 4GB is the Central Constraint
3. Core Architecture — Multi-Agent Code Orchestration
4. Agent Roles & Responsibilities
5. The "Code Change" Loop — Required Capabilities
6. Critical Challenges & Mitigations
7. Recommended Technology Stack (August 2026)
8. Minimal Viable Implementation — LangGraph + Ollama
9. Hardware Constraint Analysis — The VRAM Math
10. Viable Execution Paths, Ranked — Full Cost Breakdown
11. Shopping List — What To Actually Buy (and What Not To)
12. Total Cost of Ownership — Comparison Across 12 Months
13. Model Selection Matrix
14. Immediate Action Plan
15. Risks, Trade-offs & Alternate Considerations
16. Final Verdict & Recommendation

1 Executive Summary

Building a multi-agent system that plans, writes, reviews, tests, and commits code is fully achievable today using mature open-source frameworks (LangGraph, AutoGen) combined with strong open-weight coding models (Qwen2.5-Coder, Llama 3.1, DeepSeek-Coder-V2). None of the required software is exotic or gated — it is the same stack thousands of independent developers are using in production right now.

The one genuine blocker is local hardware. The ASUS TUF F15's RTX 1650 has only 4GB of VRAM, and a 7B-parameter model — the practical floor for reliable tool-calling and agentic coding — needs roughly 4.5GB for weights alone at 4-bit quantization, before any room is left for context (the KV cache). In plain terms: the smallest model that is actually usable for this task does not fit on this GPU, even before you load a single token of code into its context window.

The recommended fix is not a hardware upgrade. It is a **hybrid local-cloud architecture**: run the orchestration logic, state management, tool definitions, IDE, and Git operations locally (none of that needs a GPU), and send only the model inference calls to a free or near-free remote GPU over an HTTPS tunnel. This gets you access to 14B–32B parameter frontier open models at effectively \$0–\$5 of cost, with latency that is entirely acceptable for an iterative dev loop.

Bottom line: Local orchestration (LangGraph) + Remote inference (free cloud GPU or cheap pay-as-you-go API) = a working dual-agent coding loop, writing real code in your local repo, inside about 30 minutes, for a total cash outlay of \$0.

Dimension	Verdict
Software / frameworks	No blockers. LangGraph, AutoGen, Tree-sitter, LSP, Docker/E2B all free & mature.
Local GPU inference (RTX 1650 4GB)	Hard blocker for agentic coding models. Not viable even for a single 7B agent with usable context.
Local CPU offload	Technically works, practically unusable (1–3 tok/s, 5–15 min per loop).
Recommended path	Hybrid: Local orchestration + Free/cheap remote GPU inference via HTTPS tunnel.
Cost to start	\$0 (Lightning AI / Colab / Kaggle free tiers) or ~\$5 one-time (OpenRouter credits).
Time to first working loop	~30 minutes from a clean machine.

2 The Hardware Reality Check

Why the RTX 1650 4GB is the central constraint of this entire project

Every decision in this report is downstream of one number: 4GB of VRAM. It's worth understanding precisely why that number rules out local inference for this use case, rather than taking it on faith.

How VRAM gets consumed

- **Model weights** — the parameters themselves, stored at some quantization level (commonly 4-bit for consumer GPUs). A 7B model at 4-bit needs roughly 4.5GB just to sit in memory, before it does anything.
- **KV cache (context)** — every token of conversation history, file content, and tool output the model is currently "looking at" needs its own scratch memory. This grows with context length and is non-negotiable for agentic workflows, which by nature stuff a lot of file content and tool output into context.
- **Activation memory & CUDA overhead** — a few hundred MB reserved by the runtime itself, plus whatever the OS/desktop environment is already using on the same GPU.

On a 4GB card, the weights alone for a usable coding model consume all — or more than all — of the available memory, leaving 0GB or a negative budget for context. Agentic coding is precisely the workload that needs the *most* context (file contents, diffs, test output, tool results), so this is the worst possible combination for a small-VRAM GPU.

Key insight: This is not a "buy a slightly better laptop" problem. Even doubling VRAM to 8GB only barely fits a 7B model with a tiny context window — nowhere near enough for a real agentic coding loop. Meaningful local inference for this project realistically starts at 16–24GB VRAM, which is a different class of GPU (RTX 4060 Ti 16GB, RTX 3090/4090, or a cloud A10G/L4).

3 Core Architecture — Multi-Agent Code Orchestration

Three architectural patterns dominate current multi-agent coding systems. They differ mainly in how control passes between agents and how much structure is imposed on that hand-off.

Architecture	Description	Best For	Frameworks
Sequential Pipeline	Planner → Coder → Reviewer → Tester, strictly linear	Deterministic tasks (tests, docs, scripted refactors)	CrewAI, MetaGPT, LangGraph
Conversational / Chat	Shared message history; a manager agent routes turns	Exploration, debugging, open-ended architecture discussion	AutoGen (GroupChat), ChatDev
Graph / State Machine	Explicit edges, loops, conditional routing, human-in-the-loop, persistence	Production agents where reliability matters	LangGraph (primary rec.), AutoGen v0.4+

Recommendation: LangGraph. Its graph/state-machine model is the only one of the three that gives you first-class support for loops ("go back and fix the failing test"), conditional routing ("if review fails, send back to Coder; if it passes, commit"), human-in-the-loop checkpoints, and durable state persistence across runs — all of which are required, not optional, for a coding agent you'd actually trust to touch a real repository.

4 Agent Roles & Responsibilities

Role	Suggested Model	Output
Planner (Architect)	High-reasoning model — Llama 3.1 70B or Nemotron 3 Ultra	Structured plan (JSON or numbered steps)
Coder (Engineer)	Specialized code model — Qwen2.5-Coder 14B or 32B	Tool calls: read, write_patch, run_tests, grep
Reviewer / QA (Critic)	Reasoning model, given diff + test results	APPROVE or CHANGES_REQUESTED verdict
Tool Executor (Deterministic)	Plain Python — not an LLM at all	Applies patches, runs linters/tests, manages Git

Note the last row deliberately: the Tool Executor is *not* an LLM call. Applying a patch, running a test suite, or making a git commit are deterministic operations — running them through a model would only add cost, latency, and a new failure mode. Keep every step that can be plain code as plain code, and reserve model calls for genuine reasoning and generation.

5 The "Code Change" Loop — Required Capabilities

For agents to be useful beyond toy demos, they need real tool access to a repository runtime — not just chat. The table below is the minimum capability surface for a coding agent that can be trusted with an actual codebase.

Capability	Purpose	Implementation
File System Ops	Read / write / list / grep	Sandbox (Docker/VM), pathlib jail
Code Intelligence	Go-to-definition, references, call graph	Tree-sitter (local), LSP (pyright, tsserver)
Execution Environment	Run tests, lint, typecheck	Ephemeral containers — Docker, Firecracker, E2B, Modal
Version Control	Diff, commit, branch, PR	gitpython or CLI, conventional commits
Diff Application	Reliable patching without corrupting files	Unified diff + patch cmd / unidiff lib — never whole-file rewrite

Why "never whole-file rewrite" matters: Asking a model to regenerate an entire file from scratch is a common beginner mistake — it's slow, burns tokens, and silently drops code the model didn't think was relevant. Unified diffs constrain the model to expressing only the change, which is both cheaper and dramatically safer against silent data loss.

6 Critical Challenges & Mitigations

These are the failure modes that consistently derail agentic coding systems in practice, along with the standard mitigation for each.

Challenge	Why It Happens	Mitigation
Infinite / runaway loops	Coder keeps "fixing" a test it misunderstands, or Reviewer and Coder disagree indefinitely	Hard iteration caps in the graph (e.g. max 5 review cycles), then escalate to human-in-the-loop
Context window overflow	Long files, verbose tool output, and multi-turn history all compete for the same context budget	Repo maps (signatures only, via Tree-sitter) instead of full files; summarize old turns; use 32k+ context models for the Planner
Hallucinated APIs / imports	Model invents plausible-looking functions or packages that don't exist	Always run the code (typecheck + tests) before accepting; never trust unexecuted output
Bad or unparseable diffs	Small models especially struggle to emit strictly valid unified-diff syntax	Use a battle-tested diff protocol (Aider's algorithm) with automatic retry-on-failure
Cost / latency runaway	Every retry, every review cycle, every long context re-sent multiplies token spend	Cache prompts where the API supports it (DeepSeek, Anthropic); track cost per task; set a token budget per run
Security of arbitrary code execution	Agent-generated code and shell commands run with real system privileges if unsandboxed	Always execute inside ephemeral, network-restricted containers (E2B, Modal, Firecracker) — never on the host directly
Silent regressions	A patch can pass the one test it was aimed at while breaking something else	Run the full test suite, not just the touched test, before allowing a commit

7 Recommended Technology Stack (August 2026)

Layer	Recommendation	Reason
Orchestration	LangGraph (Python)	Cyclic graphs, state persistence, human-in-the-loop, streaming — the closest thing to an industry standard
Alt. Framework	AutoGen 0.4+	Better fit for free-form "society of mind" conversational patterns
Code Intelligence	Tree-sitter + LSP	Local, fast, zero cost — essential for any serious tooling
Sandbox / Runtime	E2B / Modal / Firecracker	Secure, fast cold-start, persistent filesystem
Diff / Patching	Aider's algorithm / unidiff	Battle-tested specifically for LLM-generated patches
Evaluation	SWE-bench / custom harness	Mandatory regression testing whenever prompts or models change

8 Minimal Viable Implementation — LangGraph + Ollama

A complete, runnable two-agent script (Planner → Coder, with tool use) is included below for reference. It targets an OpenAI-compatible endpoint, so the exact same code works whether the model is served locally by Ollama or remotely by a cloud GPU tunnel — only the base URL changes.

Prerequisites

```
pip install langgraph langchain-ollama langchain-core python-dotenv
# Ollama installed & running (local, or remote via tunnel)
```

two_agents.py — condensed reference (full script provided in your working files)

```
# 1. CONFIG
OLLAMA_BASE = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434/v1")
# FOR CLOUD: "https://<ngrok-id>.ngrok-free.app/v1"

# 2. MODELS (role-specific)
coder_llm = ChatOllama(base_url=OLLAMA_BASE, model="qwen2.5-coder:14b",
                       temperature=0.1, num_ctx=8192)
planner_llm = ChatOllama(base_url=OLLAMA_BASE, model="llama3.1:8b-instruct-q8_0",
                        temperature=0.0, num_ctx=8192)

# 3. TOOLS: read_file, write_file, list_files (sandboxed to ./agent_workspace)

# 4. GRAPH
# entry -> planner -> coder -> {tools -> coder | end}
# planner writes a numbered plan; coder executes it one tool call at a time
# until it emits "DONE"

# 5. RUN
# app.stream(inputs, stream_mode="values") prints each agent turn,
# then dumps the final state of every file in the workspace.
```

Changing this from local to cloud inference is a two-line change: point OLLAMA_BASE at the ngrok HTTPS URL from your cloud GPU session. Nothing else in the orchestration logic needs to change — this is exactly what makes the hybrid architecture practical.

9 Hardware Constraint Analysis — The VRAM Math

The RTX 1650, 4GB VRAM — the hard numbers

Model Size	4-bit Weights	KV Cache (4k ctx)	Total VRAM	Verdict
7B	~4.5 GB	~1.5 GB	~6.0 GB	Out of memory
3B	~2.2 GB	~0.8 GB	~3.0 GB	Fits, but zero headroom — context crash risk
1.5B	~1.1 GB	~0.5 GB	~1.6 GB	Fits, but too weak for agentic reasoning / tool use

Conclusion: You cannot run a dual-agent loop (Planner + Coder) locally on this GPU. You cannot even reliably run a single 7B coder agent with a usable context window. This isn't a configuration problem to be tuned away — it's an arithmetic fact about how much memory 4GB is.

CPU offloading (llama.cpp / Ollama) — not recommended for a dev loop

Requirement	Value
System RAM	16GB+ (DDR4/5)
Speed	1–3 tokens/sec (vs. 30–50+ tok/s on GPU)
Latency per agent turn	30–60 seconds
Latency per full loop	5–15 minutes
Practical verdict	Only useful for a one-off "hello world" sanity check. Unusable for real iterative development.

10 Viable Execution Paths, Ranked

■ Option 1 — Free Cloud GPU + Local Orchestration (Recommended)

Best developer experience, \$0 cost, frontier-class models.

Platform	GPU	Free Limit	Tunnel	Best For
Lightning AI Studios	T4 (16GB) / A10G (24GB)	22 hrs/mo	Native HTTPS (no ngrok needed)	Best overall DX; full VS Code Web IDE; persistent filesystem
Google Colab	T4 (16GB)	~12 hrs/day	Ngrok / Cloudflare	Quick prototypes; disconnects on idle
Kaggle Notebooks	T4 (16GB) / P100	30 hrs/week	Ngrok	Better session persistence than Colab
RunPod / Vast.ai	RTX 3090/4090 (24GB)	\$0.10–\$0.20/hr	SSH / HTTPS	Serious daily use; persistent volumes; "coffee money" pricing

Architecture

Local laptop (VS Code, Git, the LangGraph script) → HTTPS API calls via the OpenAI SDK → Ngrok tunnel (e.g. `https://xyz.ngrok-free.app`) → vLLM server on the cloud notebook, port 8000 → GPU (T4/L4/A10G) running Qwen2.5-Coder-14B or 32B.

What this costs in practice

- **Cash cost:** \$0 on Lightning AI / Colab / Kaggle free tiers. RunPod/Vast.ai runs roughly \$0.10–\$0.20/hr if you outgrow the free tiers.
- **Time cost:** ~15 minutes to set up the cloud session and tunnel the first time; near-zero after that.
- **Recurring friction:** Free tiers disconnect on idle and cap total hours/month — fine for daily dev sessions, annoying for always-on use.

■ Option 2 — API Credits (OpenRouter / DeepSeek / Together) — Fastest Setup

Cost: roughly \$5 buys months of development. Zero infrastructure. Access to the best models available.

Provider	Key Models	Cost / 1M Tokens	What \$5 Buys
OpenRouter	Qwen2.5-Coder-32B, Llama-3.1-70B, Nemotron 3 Ultra, GPT-4o-mini, Claude 3.5 Haiku	\$0.10 – \$0.30	20M–50M tokens
DeepSeek API	DeepSeek-Coder-V2 / DeepSeek-V2.5 (SOTA on code)	\$0.14 (cache cheaper)	Best code value per dollar
Together / Fireworks	Llama 3.1 405B, Qwen 72B	\$0.20 – \$0.60	10M–25M tokens

Code change required is minimal — swap ChatOllama for ChatOpenAI pointed at the provider's base URL and API key. No cloud notebook session to babysit, no tunnel to keep alive; this is the path with the least ongoing operational overhead.

■ Option 3 — Local CPU Offload (Ollama) — Last Resort

Only if you have zero internet access, zero budget, and extreme patience.

Requirement	Detail
System RAM	16GB+ required
Models	qwen2.5-coder:7b-q4_k_m (num_gpu 20), llama3.2:3b-instruct-q8_0
Speed	~2 tokens/sec
Full agent loop	~10 minutes

11 Shopping List — What To Actually Buy (and What Not To)

Given the recommended architecture, almost nothing needs to be purchased. This section lists the genuine options in order, so you can decide deliberately rather than defaulting to a hardware upgrade that isn't actually required.

Buy nothing (recommended starting point)

- Everything in Option 1 (Lightning AI / Colab / Kaggle free tiers) requires no purchase at all.
- A free Ngrok account (or Cloudflare Tunnel, also free) is all the networking layer you need.
- Your current laptop is already sufficient for 100% of the orchestration, IDE, and Git workflow — the GPU is only needed on the remote side.

Spend ~\$5–\$20 (cheapest way to remove all friction)

Item	Approx. Cost	Why
OpenRouter account credit	\$5 one-time	Removes the need to babysit a cloud notebook session; pay-as-you-go, no idle timeouts
DeepSeek API credit	\$5 one-time	Best raw price/performance for coding-specific tasks
Ngrok Pro (optional)	\$8/mo	Only needed if you want a stable, non-expiring tunnel URL for Option 1

Spend \$0.10–\$0.20/hr as you scale (pay-as-you-go GPU rental)

- RunPod or Vast.ai — rent an RTX 3090/4090 (24GB) by the hour once free-tier limits become a real constraint. No commitment, no hardware to own.
- This is the natural "next step up" from the free tiers, not a replacement for them — most people never need to go here for a personal dev project.

What NOT to buy right now

Tempting Purchase	Why to Hold Off
A new laptop / desktop GPU (e.g. RTX 4060 Ti 16GB)	Real money (~\$450–\$500+) for a capability the free cloud tiers already give you for \$0. Only worth it if you need offline/always-available inference for reasons beyond this project.
An eGPU enclosure for the RTX 1650 setup	Adds cost and complexity to work around a GPU that would need replacing anyway — you'd be paying twice.
A large upfront cloud API commitment	Pay-as-you-go is cheap enough here that prepaying doesn't save meaningfully — start with \$5 and top up only if you actually burn through it.

If you ever do decide to invest in local hardware (e.g. for privacy, offline work, or beyond this one project), the useful VRAM tiers are 16GB (comfortable 14B local inference) and 24GB (comfortable 32B local inference). Anything below 16GB is not really a meaningful upgrade over what you have now for agentic coding specifically.

12 Total Cost of Ownership — 12-Month Comparison

A rough comparison of realistic 12-month cost across the paths considered, assuming moderate, regular personal-project usage (a few hours of active development most days).

Path	Upfront	Monthly	12-Month Est.	Notes
Free cloud GPU (Lightning/Colab/Kaggle)	\$0	\$0	\$0	Free-tier hour limits may occasionally require waiting or switching platforms
API credits (OpenRouter/DeepSeek)	\$5	\$0–\$10	\$5–\$120	Scales with actual usage; realistic personal-project use rarely exceeds \$10–20/mo
Rented GPU (RunPod/Vast.ai)	\$0	\$15–\$60	\$180–\$720	Assumes ~3–6 hrs/day at \$0.10–\$0.20/hr; only needed past free-tier limits
New GPU purchase (16GB card)	~\$450–\$500	\$0 (+ electricity)	~\$460–\$520	Only justified by needs beyond this single project
Local CPU offload	\$0	\$0	\$0	"Free" in dollars, expensive in time — not recommended for real use

The free-tier and API-credit paths dominate on cost for a personal project by a wide margin. A hardware purchase only starts to make financial sense if usage becomes heavy and constant enough that a rented-GPU bill would exceed the purchase price within its useful life — a threshold most individual developers doing iterative coding work never reach.

13 Model Selection Matrix

Hardware / Budget	Coder Model (Primary)	Planner / Reasoner Model	Context	Est. Cost
Free cloud (T4 16GB)	Qwen2.5-Coder-14B-Instruct (4-bit)	Same model (single deploy)	8k–16k	\$0
Free cloud (L4/A10G 24GB)	Qwen2.5-Coder-32B-Instruct (4-bit)	Llama-3.1-70B (4-bit) or Nemotron 3 Ultra	32k–128k	\$0
API (\$5 budget)	DeepSeek-Coder-V2 / Qwen2.5-Coder-32B (OpenRouter)	DeepSeek-V2.5 / Llama-3.1-70B / Nemotron 3 Ultra	64k–128k	~\$0.10–\$0.30 /1M tok
Local (24GB VRAM)	Qwen2.5-Coder-32B (q4/q5)	Llama-3.1-70B (q4) / Nemotron 3 Ultra	32k+	Hardware cost
Local (16GB VRAM)	Qwen2.5-Coder-14B (q4/q8)	Llama-3.1-8B (q8) / Qwen2.5-14B-Instruct	16k–32k	Hardware cost
Local (8GB VRAM / CPU)	Qwen2.5-Coder-7B (q4)	Llama-3.2-3B / Phi-3.5-Mini	4k–8k	Very slow

Why these models

- **Qwen2.5-Coder series** — current state-of-the-art open code models, trained specifically for repo-level understanding, diff generation, and tool calling. Outperforms CodeLlama and DeepSeek-Coder-v1 on HumanEval/MBPP/SWE-bench.
- **Llama 3.1 8B/70B Instruct** — best general reasoning/instruction-following for planning and review; the 8B variant is surprisingly capable for its size.
- **Nemotron 3 Ultra (NVIDIA)** — top-tier reasoning and alignment, excellent for complex architectural decisions; needs meaningful VRAM.
- **DeepSeek-V2.5 / Coder-V2** — best price/performance via API, massive 128k context, strong coding ability.

14 Immediate Action Plan

Phase 0 — Decision (now)

Choose a path: **Option 1 (Lightning AI Studio)** is the smoothest "free tier" start. **Option 2 (OpenRouter, \$5)** is the smoothest "zero friction" start.

Phase 1 — Environment Setup (~15 minutes)

- Local: install git, Python 3.11+, uv or pip.
- Local: git clone your target repo (or create a dummy agent_workspace folder).
- Cloud: open Lightning AI Studio (or Colab). Create a Studio, select a T4 GPU, open VS Code Web.
- Cloud terminal: paste the vLLM + ngrok setup cell and run it.
- Cloud: copy the ngrok HTTPS URL (<https://xxx.ngrok-free.app/v1>).
- Local: create a .env file with OLLAMA_BASE_URL=<https://xxx.ngrok-free.app/v1>.
- Local: save two_agents.py, then run: python two_agents.py.

Phase 2 — Engineering Iteration (the real work)

Sprint	Goal	Key Tech
1	Diff tool: replace write_file with apply_unified_diff + git apply --check	unidiff, subprocess
2	Repo map: build get_repo_map tool (Tree-sitter — class/function signatures only), feed to Planner	tree-sitter, tree-sitter-languages
3	LSP tools: add go_to_definition, find_references via LSP	python-lsp-server, lsp-jsonrpc
4	Test loop: run_tests tool → capture stdout/stderr → feed back to Coder for self-correction	pytest, jest, docker
5	Evaluator: build eval_harness.py, run on ~20 real issues from your repo, measure Pass@1, cost, tokens	pytest, langsmith / wandb

15 Risks, Trade-offs & Alternate Considerations

Beyond the hardware constraint itself, a few second-order factors are worth weighing.

Free-tier reliability

Colab and Kaggle sessions disconnect on idle and reset state; Lightning AI is more persistent but still capped at ~22 hrs/month. If you need an always-on agent (not just interactive dev sessions), the free tiers stop being a good fit and either paid API credits or a rented GPU become necessary.

Data privacy

Sending proprietary code to a third-party inference provider (whether a free cloud notebook or a paid API) means that code leaves your machine. For open-source or personal projects this is usually a non-issue; for proprietary company code, check the provider's data-retention policy before routing real repositories through it, and prefer providers with an explicit no-training / no-retention guarantee for API traffic.

Model quality vs. cost trade-off

Larger models (32B+, 70B) produce noticeably more reliable diffs and fewer hallucinated APIs, but cost more per token and run slower. For a solo dev loop, the 14B–32B range is typically the sweet spot — meaningfully better than 7B, without the latency and cost of 70B+ models for every single turn.

Vendor lock-in

Because the implementation targets an OpenAI-compatible endpoint throughout, switching between Ollama, vLLM, OpenRouter, DeepSeek, or any other provider is a one-line change to the base URL. This is a deliberate design choice in the recommended stack and should be preserved rather than hard-coding any single provider's SDK.

When a hardware upgrade does make sense

If, after a month or two of real use, GPU rental or API spend is consistently exceeding roughly \$25–\$30/month, or if offline/low-latency local inference becomes a hard requirement, that is the point to revisit a 16GB+ GPU purchase — evaluated then against actual observed usage rather than guessed upfront.

16 Final Verdict & Recommendation

Your RTX 1650 4GB is not a blocker for this project — only for local inference specifically.

Adopt the hybrid architecture: **local orchestration (LangGraph) + remote inference (free cloud GPU or a cheap pay-as-you-go API)**. This is the standard workflow used by the large majority of independent agentic-coding developers today — it is not a workaround or a compromise, it is simply how this class of project is normally built when a workstation-class GPU isn't available.

Start with **Lightning AI Studio** (free tier) or **OpenRouter** (\$5 credit) today. Following the Phase 1 setup steps above, a working dual-agent loop writing real code into your local repository is achievable within about 30 minutes, at a total cost of \$0.

Recommended path at a glance

Step	Action
1	Sign up for Lightning AI Studio (free) and OpenRouter (add \$5 credit) — both, so you have a fallback.
2	Set up the local repo, Python environment, and the two_agents.py LangGraph script.
3	Start a Lightning AI GPU session, run the vLLM + tunnel setup, point the local script at it.
4	Run the dual-agent loop against a small, real task in your repo to validate the pipeline end-to-end.
5	Work through Sprints 1–5 (diff tooling, repo map, LSP, test loop, evaluator) to harden it for daily use.